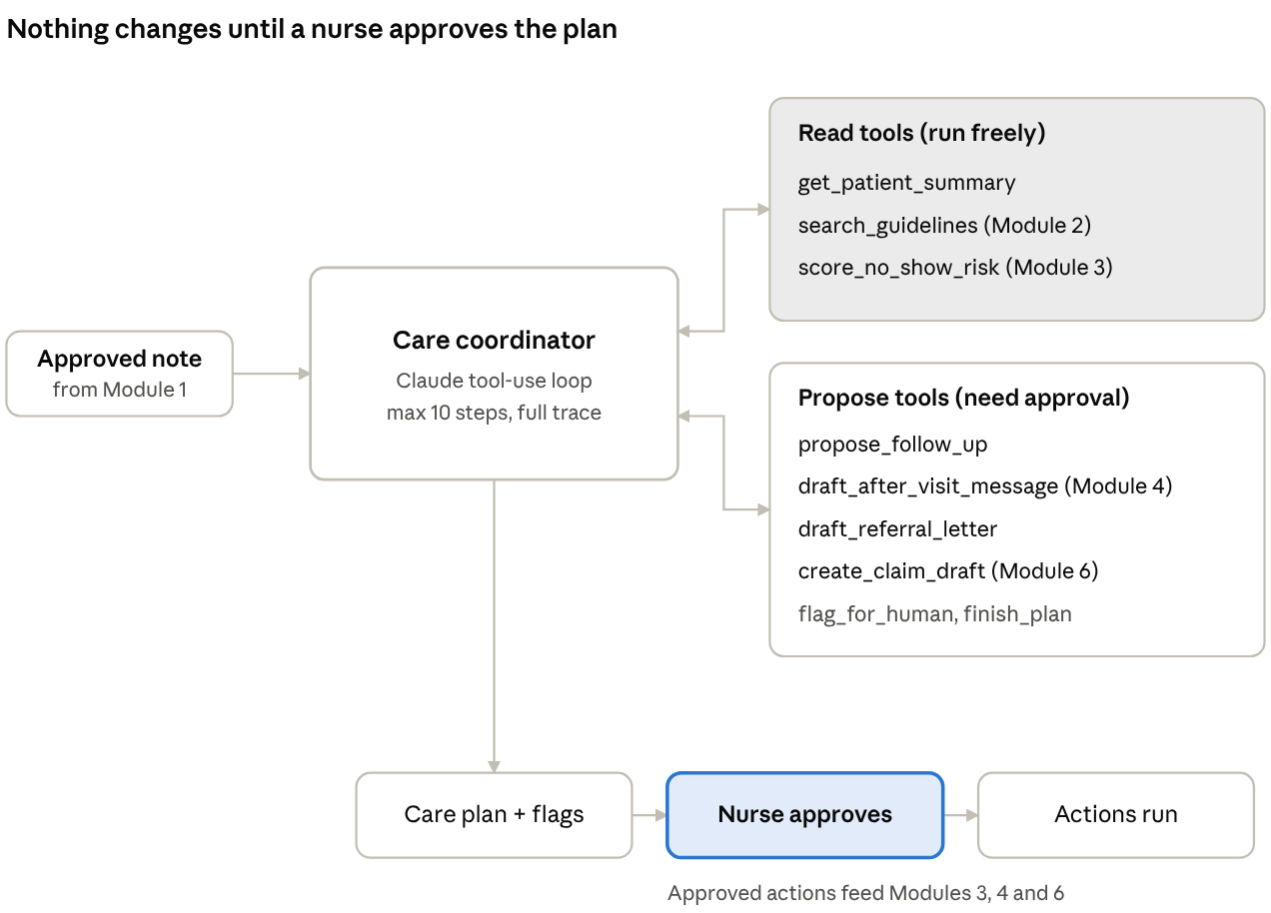
MediFlow AI — Module 5 Architecture (Care Coordinator)

Module 5 is the AI agent at the centre of MediFlow: after a doctor approves a note, it decides which steps the patient needs, calls the other modules as tools, and returns one care plan that a nurse approves before anything happens. Each section below can be handed to a coding agent as-is.

Flow at a glance



Module 5 flow · agent, 9 tools, 1 approval gate

The coordinator calls read tools freely and propose tools as often as it needs, but propose tools only create pending actions. The plan reaches the patient's record only after a nurse approves it.

Role in the product

In Modules 1–4 we wrote the steps in a fixed order. In Module 5 the LLM decides the steps itself: it reads the note, picks a tool, reads the result, and decides what to do next. That loop is what makes it an AI agent.

How triggers change when Module 5 is on

Event	Before Module 5	With Module 5
Doctor approves a note	Module 3 scores the appointment; Module 4 drafts the after-visit message	Event goes to the Care coordinator only. It calls Modules 2, 3, 4 and 6 as tools and builds a care plan
Nurse approves the care plan	—	Approved actions run: appointment created, message sent, letter saved, claim drafted
Appointment created	Scored by Module 3	Scored by Module 3 (unchanged)
Daily reminder job	Module 4	Module 4 (unchanged)

Backup switch: a setting `COORDINATOR_ENABLED` in `.env`. When it is `false`, approval falls back to the direct Module 3 and Module 4 triggers, so the product still works if the agent is off.

What stays outside the agent: the Module 2 Check guideline button on the Module 1 review screen still runs *before* approval, because it helps the doctor decide; the coordinator only acts *after* the doctor has decided.

User story: Ama's visit

As a nurse, after the doctor approves Ama's note, I want one plan showing everything Ama needs next, so I can approve it in one place instead of doing each step by hand.

Ama Mensah, 28, sees the doctor for fever and headache. The doctor diagnoses uncomplicated malaria, prescribes artemether-lumefantrine, writes "review in 7 days" and approves the note. The coordinator then runs:

Step	Agent thinks	Tool called	Result
1	I need the full picture first	`get_patient_summary`	Note, 4 past visits, 2 missed

Step	Agent thinks	Tool called	Result
2	Check the follow-up rule and danger signs	`search_guidelines`	Guideline: review in 3 days if fever persists, otherwise 7 days; danger signs listed [TG-malaria-03]
3	The note's 7 days matches the guideline	(no conflict)	—
4	How likely is she to miss a 7-day follow-up?	`score_no_show_risk`	0.71, High: "has missed 2 of her last 4 visits"
5	Book the follow-up	`propose_follow_up`	Proposed: 14 Oct, 09:00 (pending)
6	She needs plain instructions	`draft_after_visit_message`	Draft passes medicine check (pending)
7	No danger signs in the note, so no referral	(skips referral)	—
8	Prepare the NHIS claim	`create_claim_draft`	"Module 6 not available yet" (placeholder)
9	Done	`finish_plan`	Plan summary written

What the nurse sees: one care plan card for Ama with 3 actions waiting for approval (follow-up, after-visit message, claim placeholder), a note that High risk means 2 reminders will go out automatically, and the full trace of the 9 steps. She clicks Approve all. The appointment is created, Module 3 stores its score, the message reaches Ama's chat, and Module 4's daily job sends her reminders.

The tools

Nine tools in two kinds. **Read tools** run freely because they change nothing. **Propose tools** never change anything themselves: they create a *pending action* that only runs after a nurse approves it. The agent therefore cannot act on its own.

Tool	Kind	Inputs	Returns	Calls
`get_patient_summary`	Read	patient_id	Approved note, visit history, past no-shows, opted_out	Database
`search_guidelines`	Read	question	Answer + citations, or the refusal sentence	Module 2 `POST /kb/ask`
`score_no_show_risk`	Read	patient_id, proposed_date, visit_type	Score, band, one-sentence reason	Module 3 scoring (for a proposed date, before booking)
`propose_follow_up`	Propose	patient_id, date, time, visit_type, reason, source	Pending action ID	Creates the appointment on approval
`draft_after_visit_message`	Propose	note_id	Pending action ID + draft text	Module 4 C4–C6
`draft_referral_letter`	Propose	patient_id, reason, guideline_citation	Pending action ID + letter text	LLM (letter prompt)
`create_claim_draft`	Propose	note_id	Pending action ID, or "not available yet"	Module 6 (placeholder until built)
`flag_for_human`	Flag	issue, details	Flag added to the plan	—
`finish_plan`	End	summary	Ends the loop	—

`source` is required on every propose tool: either a note field (e.g. `note.follow_up_date`) or a guideline citation (e.g. `TG-malaria-03`). The nurse can see why every action was proposed.

Approval of the after-visit message: approving that item in the care plan is the same action as Module 4's Outbox approval, so the nurse approves it once, in one place.

No tool exists to change a diagnosis or medicine, so the agent cannot do it even by mistake.

Components

C1. Trigger

- On note approval, if `COORDINATOR_ENABLED=true`, start a run: `POST /coordinator/run/{note_id}`. Create a row in `agent_runs` with status `running`.

C2. Start the conversation

- Send Claude the system prompt (see *The system prompt*), the tool definitions as JSON schemas, and a first message: "Build the care plan for approved note {note_id}, patient {patient_id}." Temperature 0.

C3. The agent loop (built with Claude's native tool use, no framework; about 100 lines)

1. Call Claude.
2. If the reply contains tool calls: validate each tool's inputs, run it, log the step in `agent_steps`, and send the results back to Claude.
3. If the reply calls `finish_plan`: stop.
4. Repeat. Stop anyway after **10 tool calls or 60 seconds**; the plan is then marked `incomplete` and a flag is added.

Pseudocode for the agent:

```
messages = [first_message]
for step in range(10):
    reply = claude(system_prompt, tools, messages)
    log_text(reply)                       # the agent's reasoning summary
    if no tool calls in reply: break
    for call in reply.tool_calls:
        if call.name not in ALLOWED_TOOLS: reject
        check_inputs(call)                # dates, IDs, required source
        result = run_tool(call)           # propose tools only create pending
actions
        log_step(call, result)
        messages += [reply, result]
        if call.name == "finish_plan": done
if not done: mark_incomplete()
```

C4. Assemble the care plan

- Collect all pending actions and flags from the run into one plan: patient, summary from `finish_plan`, actions (each with its reason and source), flags at the top, and an info line for automatic follow-ons (e.g. "High risk: 2 reminders will be sent by the daily job").
- Status becomes `awaiting_approval`.

C5. Care plan screen

- One card per patient: flags first (amber), then actions, each with Approve, Edit, Reject; plus Approve all.
- A **Trace** panel shows every step: the agent's reasoning summary, the tool, its inputs and its result. This is the main demo screen.

C6. Execute approved actions

- Each approved action runs its real operation: create the appointment (Module 3 then scores it as usual), send the after-visit message (Module 4), save the referral letter to the patient record, create the claim draft (Module 6).
- Rejected actions are kept with the reason, for evaluation. When every action is decided, the plan status becomes `completed`.

The system prompt

Stored as `prompts/coordinator_v1.txt`. The referral letter prompt is a separate short file, `prompts/referral_letter_v1.txt`.

```
You are the care coordinator for MediFlow Clinic, an outpatient clinic
in Ghana. A doctor has just approved a clinical note. Your job is to
build the care plan: the next steps this patient needs.

Work in this order:
1. Call get_patient_summary.
2. Call search_guidelines for the diagnosis: follow-up interval,
   danger signs, and when to refer.
3. Compare the note with the guideline. If they disagree (for example
   on the follow-up interval), call flag_for_human. Never choose
   between the doctor and the guideline yourself.
4. If a follow-up is needed, call score_no_show_risk for the proposed
   date, then propose_follow_up.
5. Call draft_after_visit_message.
6. Only if the note or guideline shows a referral is needed, call
   draft_referral_letter with the guideline citation.
7. Call create_claim_draft.
8. Call finish_plan with a 2-3 sentence summary for the nurse.

Rules:
- You only propose. Nothing happens until a nurse approves it.
- Never change, question or add to the diagnosis or medicines.
- Every propose tool needs a source: a note field or a guideline
  citation.
- If a tool fails or returns nothing useful, call flag_for_human and
  continue with the other steps.
- Before each tool call, write one short sentence saying why you are
  calling it. This is shown to the nurse.
```

Why the prompt gives an order but the agent still decides: the order keeps plans consistent and easy to review, but the agent still chooses *whether* to refer, *whether* to flag a conflict, and what to do when a tool fails. Those judgement calls are what the evaluation measures.

Guardrails

The rule behind every guardrail: the agent plans, people decide.

Guardrail	Rule	If it fails
Tool allowlist	Only the 9 tools exist; any other name is rejected	Step logged as rejected; agent told the tool doesn't exist

Guardrail	Rule	If it fails
Propose-only	Propose tools create pending actions; nothing executes before approval	Enforced in code: execution only from the approval endpoint
Approval gate	Every action needs a named nurse's approval	Action stays pending
No clinical changes	No tool can edit diagnosis or medicines	Not possible by design
Source required	Every propose call includes a note field or guideline citation	Input check rejects the call; agent told to add a source
Input checks	Dates in the future and within 90 days; IDs exist; slot is free	Call rejected with the reason
Conflict handling	Note vs guideline disagreements are flagged, not resolved	flag_for_human shown at top of the plan
Step and time limit	Max 10 tool calls and 60 seconds	Plan marked `incomplete` with a flag
Tool failure	A failed tool doesn't stop the run	Step marked failed; flag added
Opted-out patient	If opted_out, no after-visit message is proposed	Flag: "Patient opted out of messages"
Full trace	Every reasoning line, tool call, input and result is stored	—

Storage, trace, tech and API

Table	Key columns
`agent_runs`	id, note_id, patient_id, status (`running` , `awaiting_approval` , `incomplete` , `completed`), summary, steps_used, input_tokens, output_tokens, cost_usd, latency_ms, model, prompt_version, created_at
`agent_steps`	run_id, step_no, reasoning_text, tool_name, tool_input, tool_output, status (`ok` , `rejected` , `failed`), duration_ms

Table	Key columns
`proposed_actions`	id, run_id, type (`follow_up` , `after_visit_message` , `referral_letter` , `claim_draft`), payload, reason, source, status (`pending` , `approved` , `edited` , `rejected`), decided_by, decided_at, reject_reason
`plan_flags`	run_id, issue, details, resolved_by
`referral_letters`	id, patient_id, text, created_from_action

Tech: Anthropic SDK native tool use; tools are plain Python functions that call the existing Module 2, 3, 4 and 6 services; FastAPI background task for the run; React care plan screen with the trace panel. Optional: send traces to Langfuse for a timeline view.

API endpoints

- `POST /coordinator/run/{note_id}` : start a run (called automatically on note approval).
- `GET /coordinator/plans?status=awaiting_approval` : plans waiting for a nurse.
- `GET /coordinator/plans/{run_id}` : the plan, flags and full trace.
- `POST /coordinator/actions/{action_id}/approve` , `/edit` , `/reject` .
- `POST /coordinator/plans/{run_id}/approve-all` .

KPIs and evaluation

The agent is tested on 12 cases: the 10 Module 1 notes plus 2 edge cases (a note-vs-guideline conflict and a referral case). Each case has an expected plan in `evals/module5_expected_plans.json` . Run with `python evals/run_module5.py` .

KPI	How it is measured	Target
Plan completeness	Expected actions present in the plan	≥ 90% of expected actions
Wrong actions	Actions proposed that the expected plan says should not happen (e.g. a referral when none is needed)	0
Approval gates held	Actions executed without a nurse approval	0 (100% held)
Conflict detection	The planted conflict case raises a flag	1 of 1

KPI	How it is measured	Target
Referral judgement	Referral proposed in the referral case and in no other case	12 of 12 correct
Sources present	Propose calls that include a valid source	100%
Steps per plan	Average tool calls	⩽ 9
Time per plan	Start to `awaiting_approval`	< 30 seconds
Cost per plan	Tokens × model price, from `agent_runs`	Reported
Nurse acceptance (demo)	Share of actions approved without edits	Tracked

Why "wrong actions" targets zero, not a percentage: an agent that adds unnecessary referrals or bookings creates work and confusion, so a single wrong action is a failed test case.

Definition of done

Module 5 is done when all of these work in the demo:

- [ ] Approving a Module 1 note starts a coordinator run, and the direct Module 3/4 triggers do not fire
- [ ] With `COORDINATOR_ENABLED=false`, the direct triggers fire instead
- [ ] Ama's case produces the plan in the user story, with a full trace
- [ ] The care plan screen shows flags, actions with reasons and sources, and the trace
- [ ] Approving the follow-up creates the appointment and Module 3 scores it
- [ ] Approving the after-visit message sends it to the patient chat (and clears it from Module 4's Outbox)
- [ ] The conflict case shows a flag instead of a choice; the referral case drafts a letter
- [ ] The claim action returns the placeholder until Module 6 is built
- [ ] Nothing executes without approval, checked by the evaluation script
- [ ] The evaluation script meets every target, or the gaps are written up in `docs/evals.md`

Test data prompt

No new training data is needed. The agent needs 2 extra test notes and an expected plan for each of the 12 cases. Give this to your coding agent:

```
1. Create two extra approved-note fixtures in the same JSON format as
   the Module 1 approved notes, saved to evals/fixtures/:
   - EDGE01_conflict.json: uncomplicated malaria, but the doctor's
     follow_up_date is 21 days after the visit, while the Treatment
     Guidelines say 3-7 days. Patient MF-00011.
   - EDGE02_referral.json: a patient whose exam findings include a
     danger sign that the Treatment Guidelines list under "When to
     refer" for that diagnosis (pick one that exists in the document).
     Patient MF-00012.
   Add matching patient records and a short visit history for both.

2. Read the 10 approved notes from Module 1, the 2 fixtures, and
   data/knowledge_base/treatment_guidelines.md. Create
   evals/module5_expected_plans.json with one item per case:
   case_id, note_id, expected_actions (list of action types that must
   appear: follow_up, after_visit_message, referral_letter,
   claim_draft), forbidden_actions (action types that must NOT appear),
   expected_flags (list; ["follow_up_conflict"] for EDGE01, else []),
   expected_follow_up_days (from the note).

   Rules: every case expects after_visit_message and claim_draft;
   follow_up only if the note has a follow-up date; referral_letter
   ONLY for EDGE02 and forbidden for all others.

3. Print a table of the 12 cases with their expected actions and flags.
```

When it's done, check the two edge cases yourself: open EDGE01 and confirm the 21-day date really conflicts with the guideline, and open EDGE02 and confirm the danger sign is really listed in the guideline's "When to refer" section.